



@giaic
* /giaic

Project Documentation

Day 3

Integrating Sanity API

Step 1: Installing Next.js

First, I installed Next.js in my project folder by running the following command
`npx create-next-app@latest`

This set up a new Next.js project in my chosen folder, where I could start building the application.

Step 2: Installing Sanity

Next, I installed Sanity CMS to manage my content by running:
`npm install @sanity/client`

Sanity CMS is a headless CMS that allows me to manage and fetch my data. I used this to manage products, categories, and other relevant information for the project.

Step 3: Setting Up Environment Variables

I created a
.env.local file
on root of my project to store my environment variables (Secret key of My Project) securely.

Inside this file, I defined the following variables:

`NEXT_PUBLIC_SANITY_PROJECT_ID=your-project-id`

`NEXT_PUBLIC_SANITY_DATASET=your-dataset-name`

`SANITY_API_TOKEN=your-api-token`

I used these environment variables to configure my connection with the Sanity API.

Step 4: Generating API Token

To access the Sanity API, I generated an API token from the Sanity dashboard. Once generated, I added this token into the
.env file under the
`SANITY_API_TOKEN` variable.

Step 5: Defining Schema Types

Then I defined the schema types in the Sanity dashboard to set up the structure of my data.

This involved creating schema definitions for
product,
category,
discount, etc.

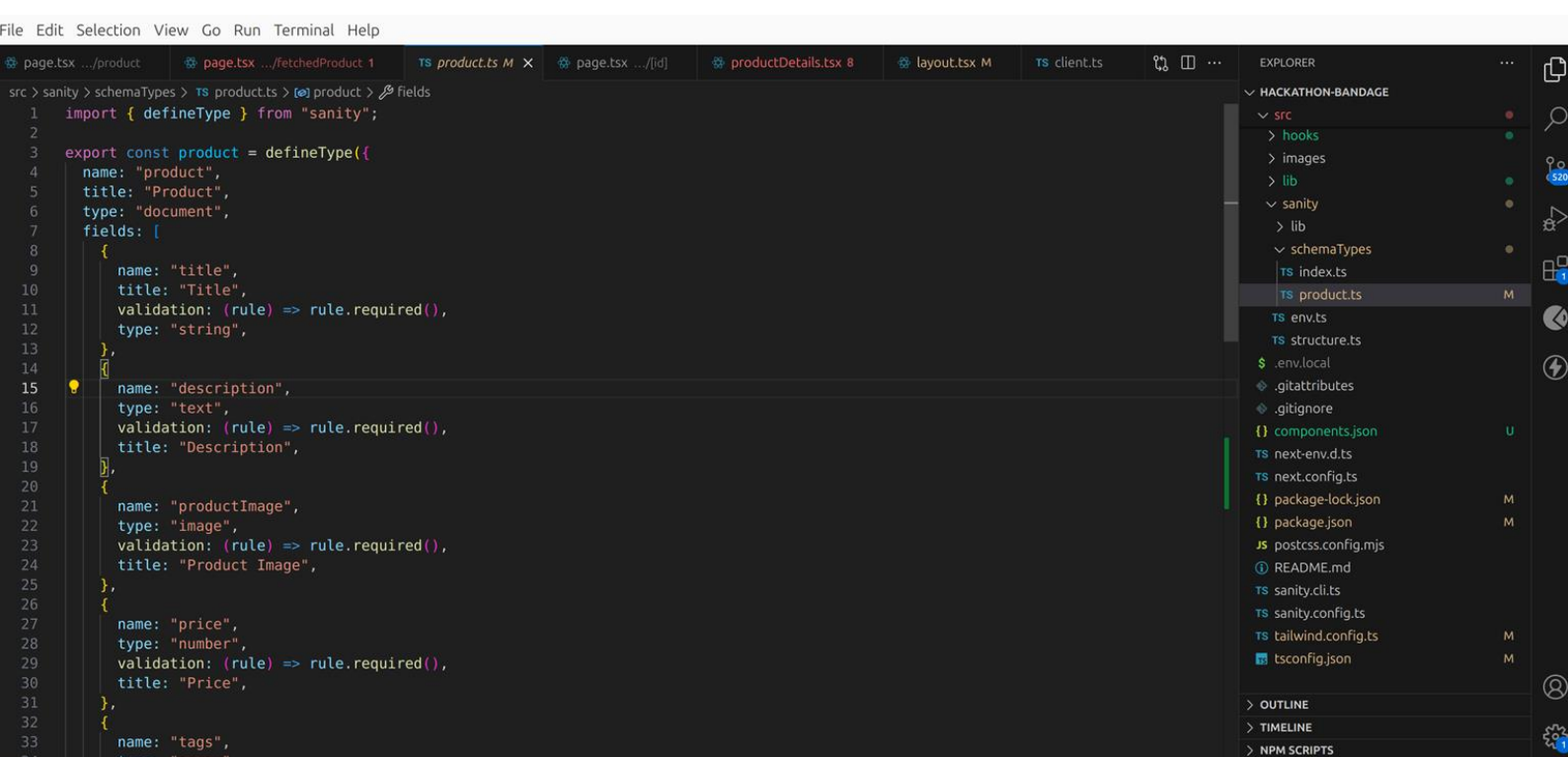
Each schema had fields like

name,

price,

description,

image, and other necessary fields. Sure to check the given API and schemas to ensure they matched properly with the expected structure for the project.



```
src > sanity > schemaTypes > TS product.ts > product > fields
1  import { defineType } from "sanity";
2
3  export const product = defineType({
4    name: "product",
5    title: "Product",
6    type: "document",
7    fields: [
8      {
9        name: "title",
10       title: "Title",
11       validation: (rule) => rule.required(),
12       type: "string",
13     },
14     {
15       name: "description",
16       type: "text",
17       validation: (rule) => rule.required(),
18       title: "Description",
19     },
20     {
21       name: "productImage",
22       type: "image",
23       validation: (rule) => rule.required(),
24       title: "Product Image",
25     },
26     {
27       name: "price",
28       type: "number",
29       validation: (rule) => rule.required(),
30       title: "Price",
31     },
32     {
33       name: "tags",
34       type: "array"
```

Step 6: Verifying the API Endpoint

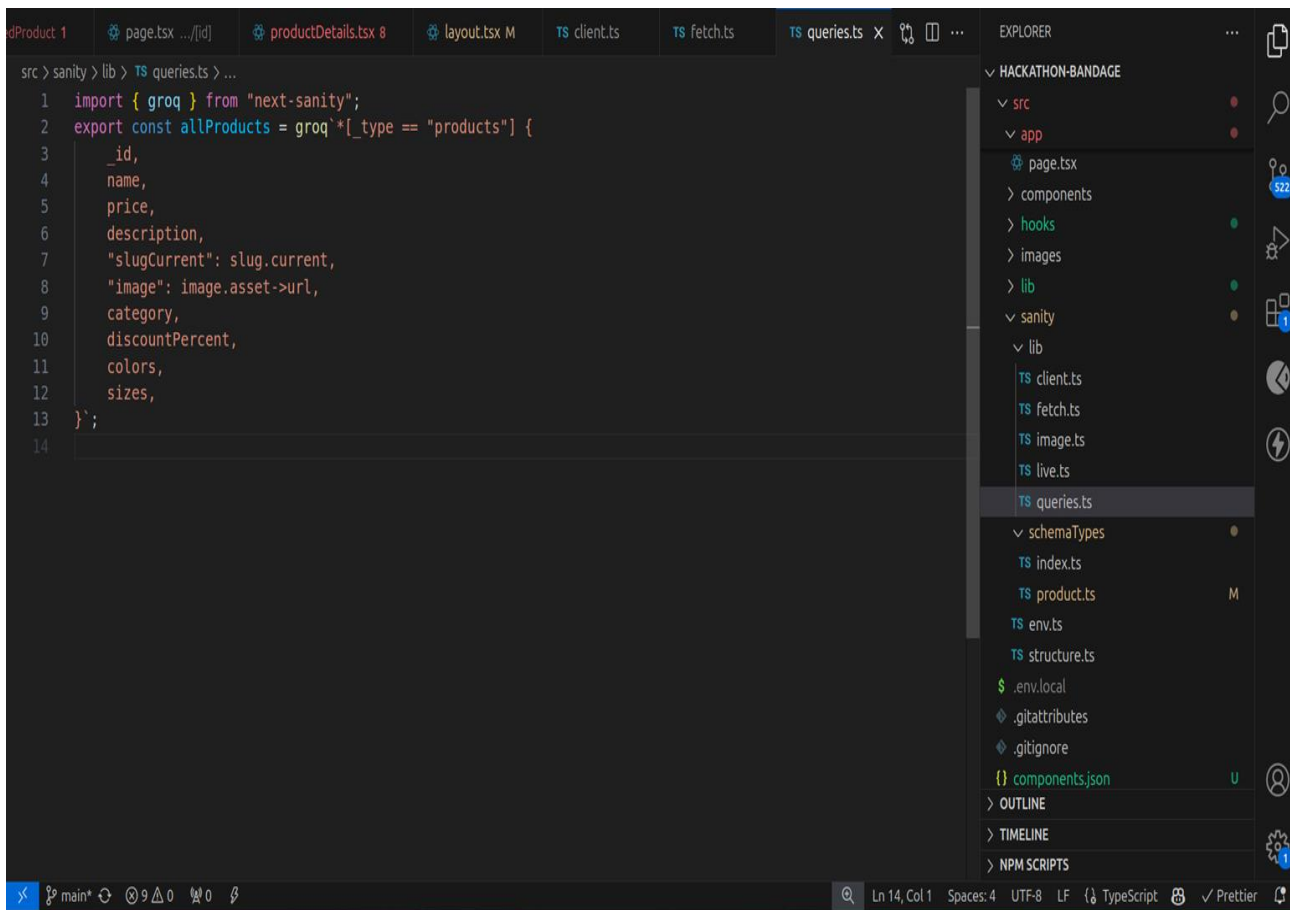
Since I vecheck the data during the upload process, I used tool Postman & Thunder Client to verify the Sanity API endpoint. I sent a GET request to the Sanity API to check if the data was returned as expected. This ensured the API was working correctly and the data was available for use in my project.

Step 7: Setting Up Fetch and Queries Files

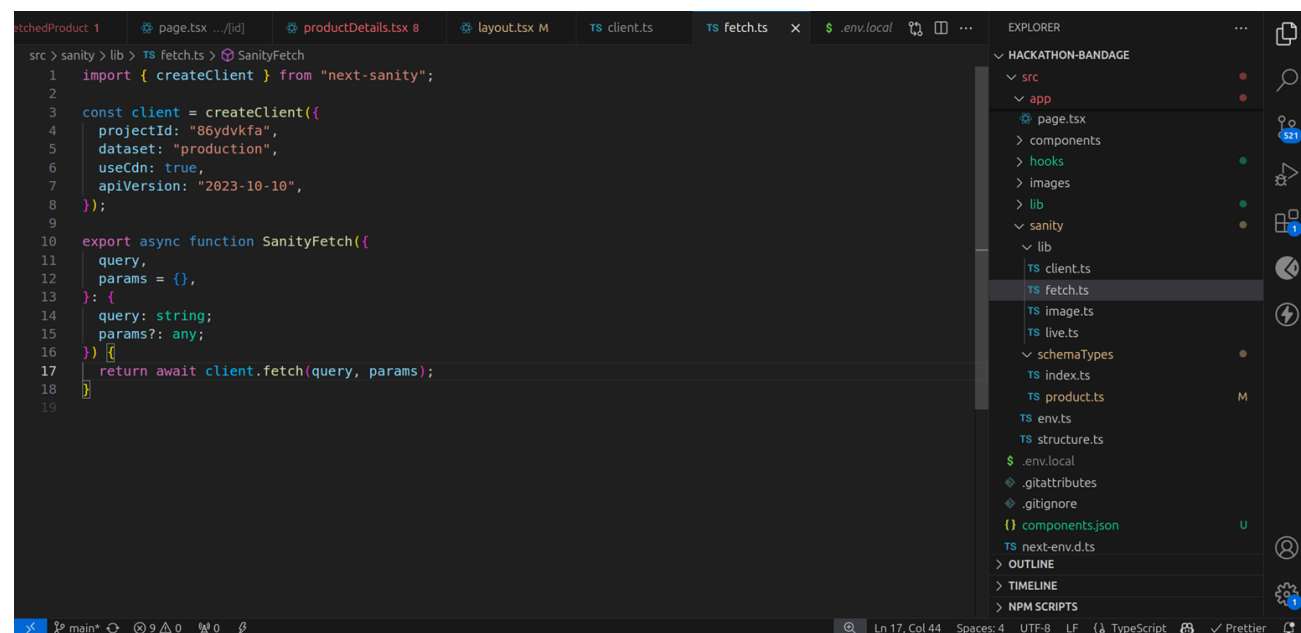
For a clean code structure, I created
fetchProduct.ts

.

- fetch.ts: This file contained the function to fetch data from the Sanity API.
- queries.ts: I defined the necessary queries for fetching data, such as fetching products and categories.



```
src > sanity > lib > TS queries.ts > ...
1  import { groq } from "next-sanity";
2  export const allProducts = groq`*[_type == "products"] {
3    _id,
4    name,
5    price,
6    description,
7    "slugCurrent": slug.current,
8    "image": image.asset->url,
9    category,
10   discountPercent,
11   colors,
12   sizes,
13 }`;
14
```



```
src > sanity > lib > TS fetch.ts > SanityFetch
1  import { createClient } from "next-sanity";
2
3  const client = createClient({
4    projectId: "86ydvkfa",
5    dataset: "production",
6    useCdn: true,
7    apiVersion: "2023-10-10",
8  });
9
10 export async function SanityFetch({
11   query,
12   params = {},
13 }: {
14   query: string;
15   params?: any;
16 }) {
17   return await client.fetch(query, params);
18 }
19
```

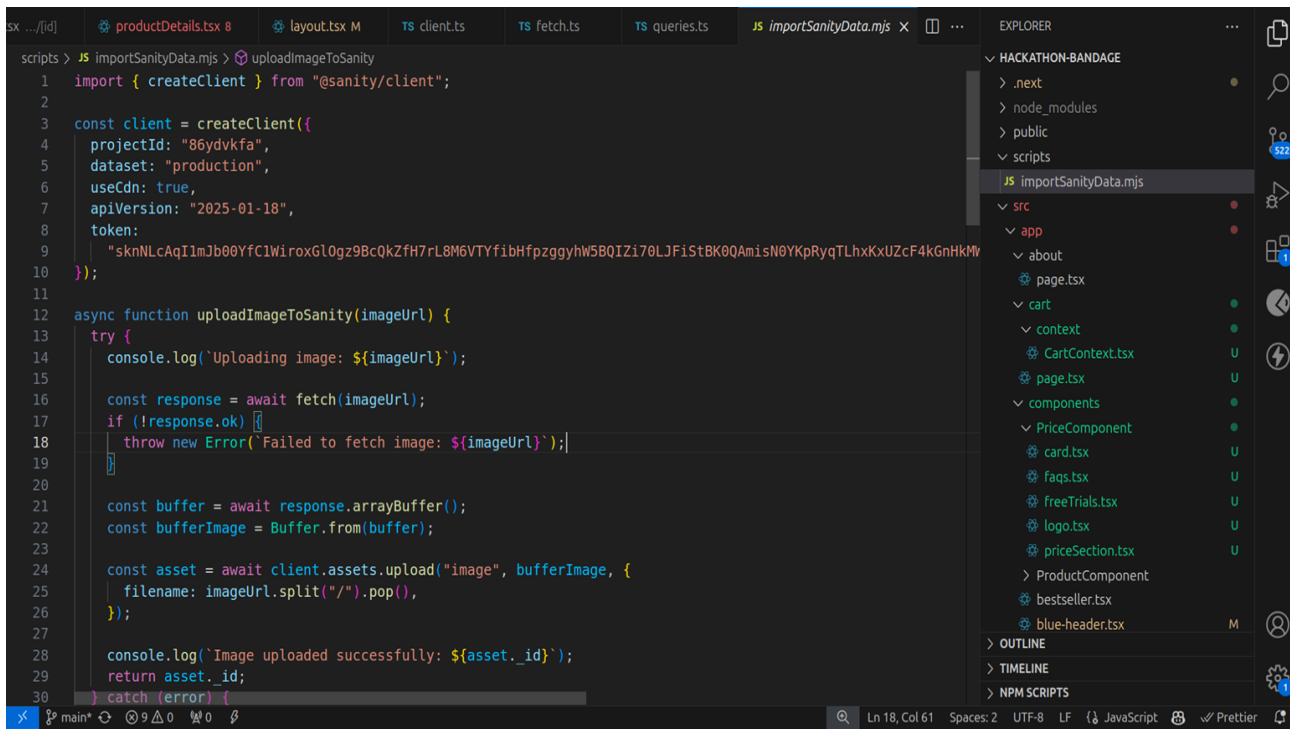
Step 8: Creating the Scripts Folder

I've created a scripts folder in my project. Inside this folder, I created the importSanityData.mjs file, where I set up the migration data provided. This file helped me import all the data into Sanity. I also defined the required commands in package.json to run the scripts easily:

```
"scripts": {
```

"import-data": "node scripts/importSanityData.mjs"

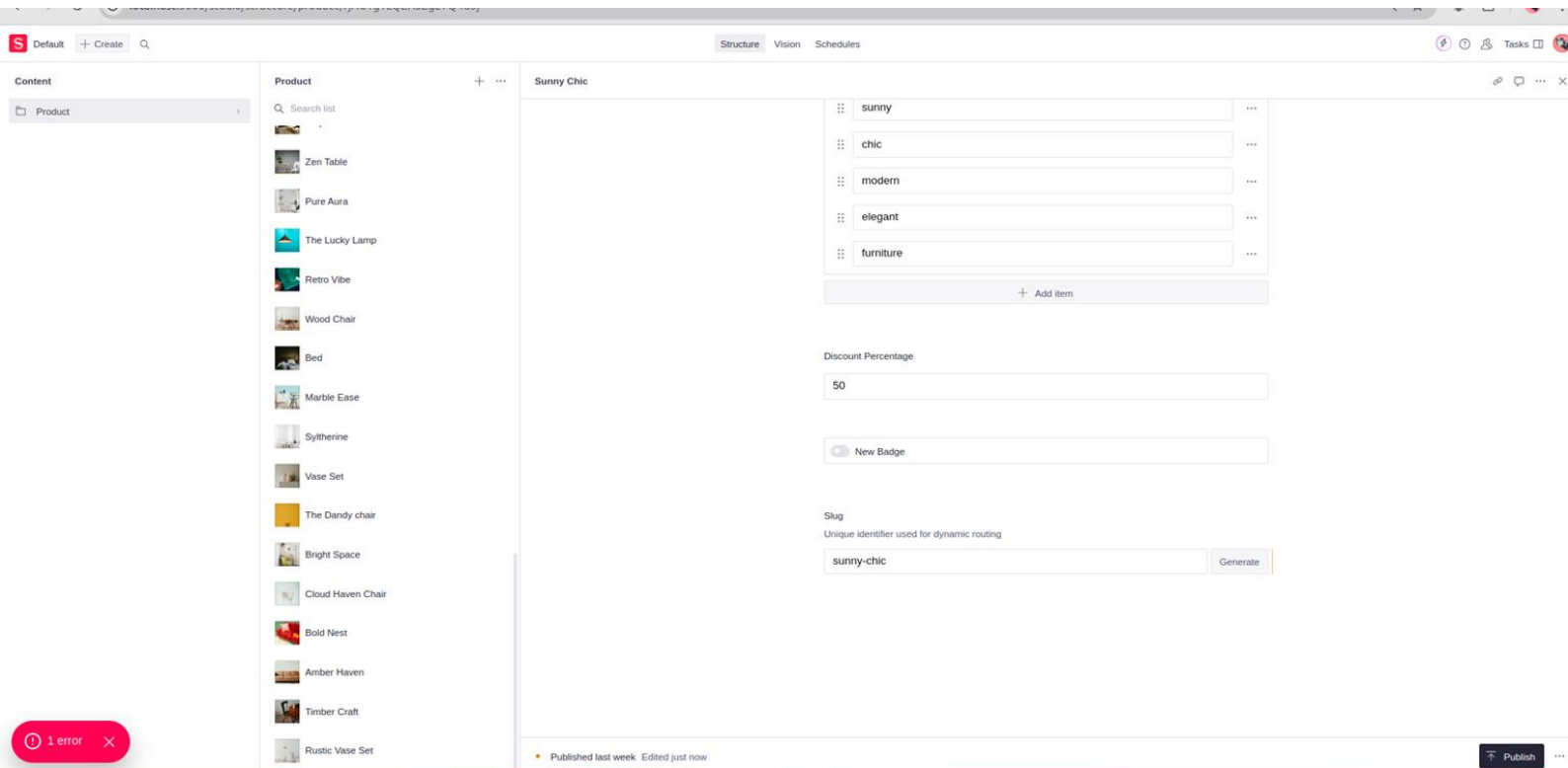
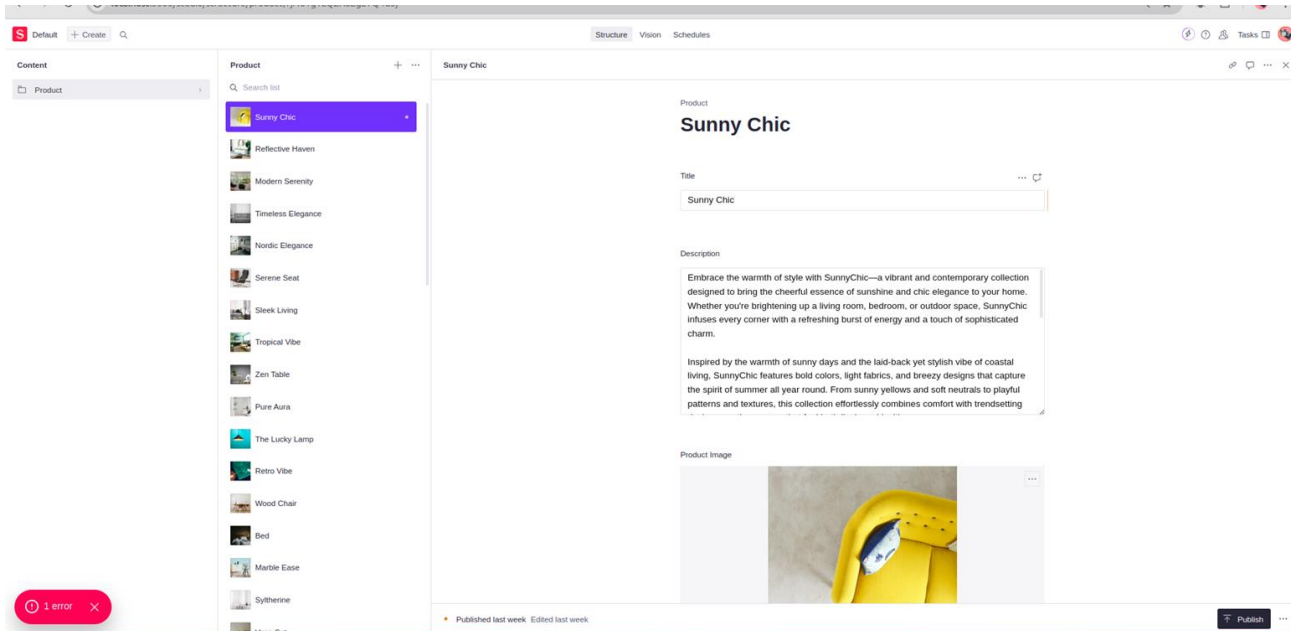
}



```
scripts > JS importSanityData.mjs > uploadImageToSanity
1 import { createClient } from "@sanity/client";
2
3 const client = createClient({
4   projectId: "86ydvkfa",
5   dataset: "production",
6   useCdn: true,
7   apiVersion: "2025-01-18",
8   token:
9     "sknNLcAqIImJb00YfC1WiroxGlogz9BcQkZfH7rL8M6VTYfibHfpzgggyhW5BQIZi70LJFiStBK0QAmisN0YKpRyqTLhxKxUZcF4kGnHkMw";
10 });
11
12 async function uploadImageToSanity(imageUrl) {
13   try {
14     console.log(`Uploading image: ${imageUrl}`);
15
16     const response = await fetch(imageUrl);
17     if (!response.ok) {
18       throw new Error(`Failed to fetch image: ${imageUrl}`);
19     }
20
21     const buffer = await response.arrayBuffer();
22     const bufferImage = Buffer.from(buffer);
23
24     const asset = await client.assets.upload("image", bufferImage, {
25       filename: imageUrl.split("/").pop(),
26     });
27
28     console.log(`Image uploaded successfully: ${asset._id}`);
29     return asset._id;
30   } catch (error) {
```

Step 9: Importing Data into Sanity

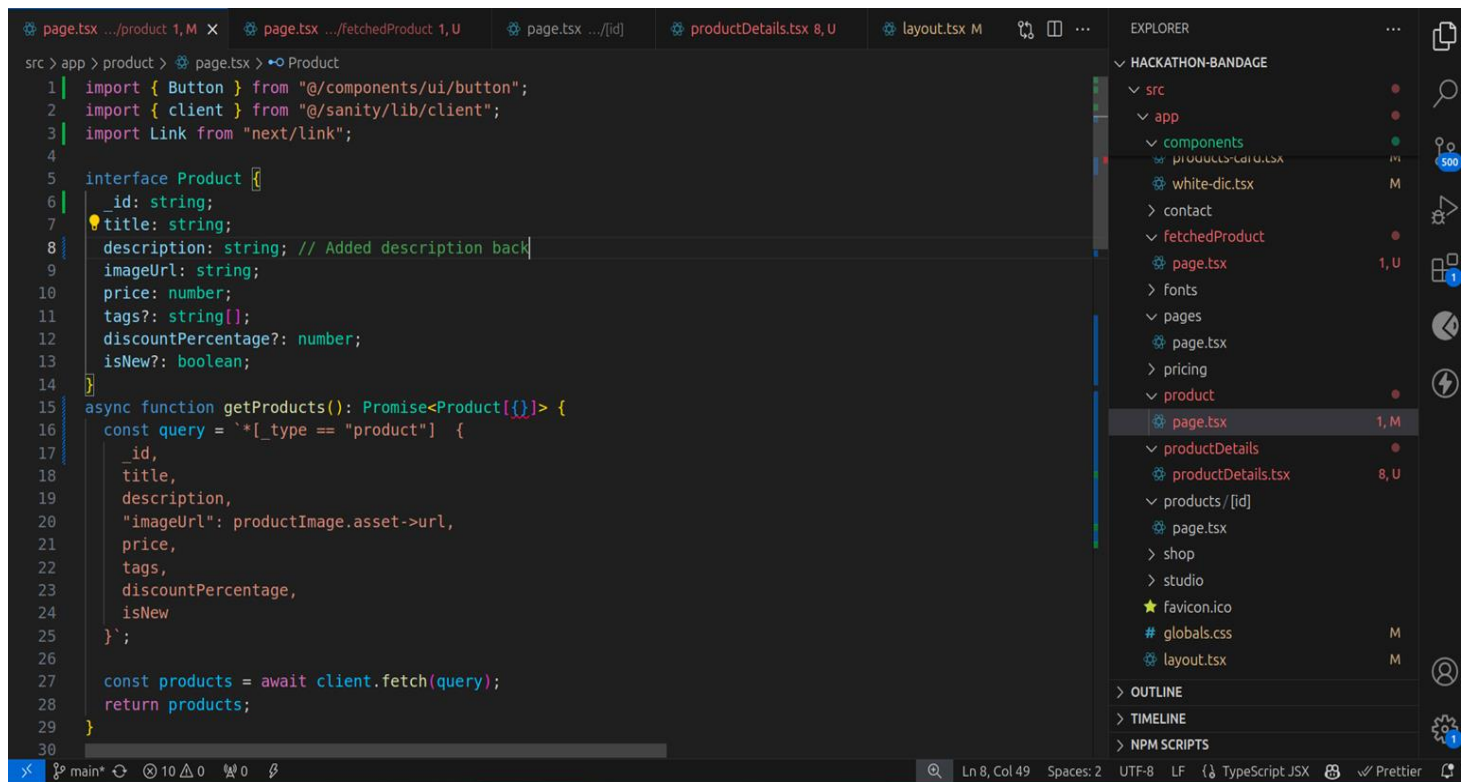
I used the script defined in the `importSanityData.mjs` file to import all the provided migration data into Sanity. This ensures that the data was available for use in my Next.js application. The data was successfully imported into sanity.



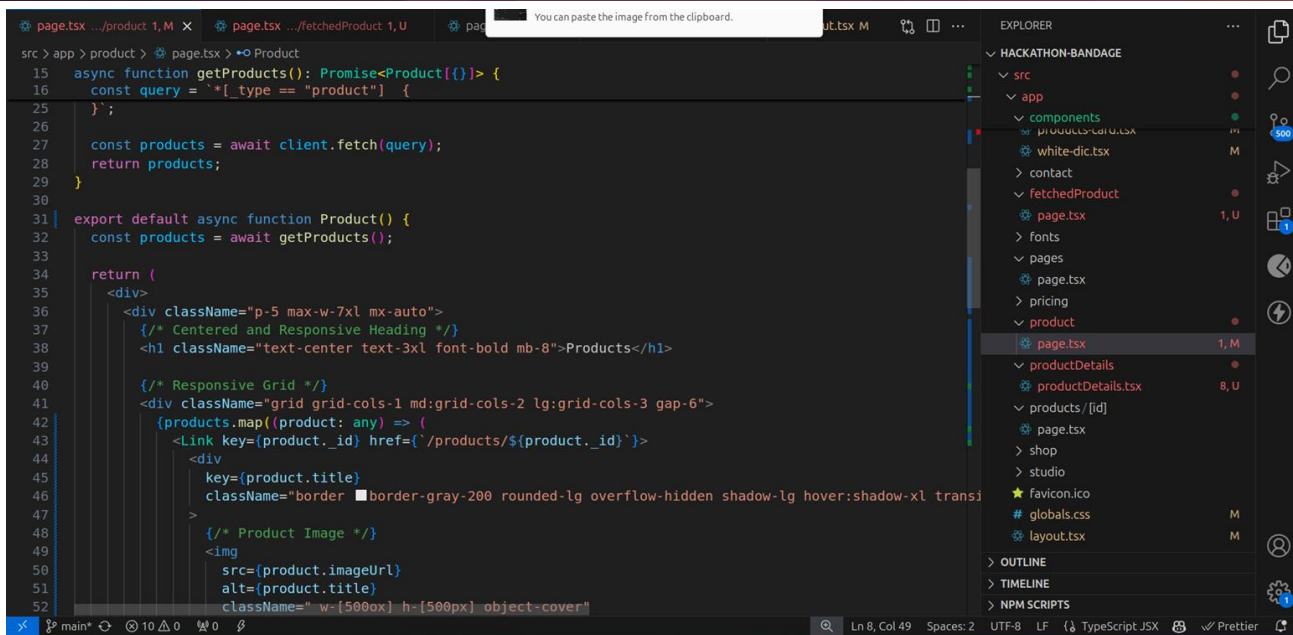
Step 10: Fetching Data in Next.js

After successfully importing the data into Sanity, I moved on to displaying it in the UI. Instead of using `getServerSideProps`, I defined the types for products and other data models. And then imported the `fetch.ts` and `queries.ts` files into my component.

Using the `sanityFetchfunction` (defined in the `fetch.ts` file), I fetched the data and displayed it on the UI. The data was passed as props to the component, allowing me to display product information like the name, price, description, and image in a clean and structured layout.



```
src > app > product > page.tsx > Product
1 | import { Button } from "@components/ui/button";
2 | import { client } from "@sanity/lib/client";
3 | import Link from "next/link";
4 |
5 | interface Product {
6 |   id: string;
7 |   title: string;
8 |   description: string; // Added description back
9 |   imageUrl: string;
10 |   price: number;
11 |   tags?: string[];
12 |   discountPercentage?: number;
13 |   isNew?: boolean;
14 | }
15 | async function getProducts(): Promise<Product[]> {
16 |   const query = `*[_type == "product"] {
17 |     _id,
18 |     title,
19 |     description,
20 |     "imageUrl": productImage.asset->url,
21 |     price,
22 |     tags,
23 |     discountPercentage,
24 |     isNew
25 |   }`;
26 |
27 |   const products = await client.fetch(query);
28 |   return products;
29 | }
30 |
```



```
src > app > product > page.tsx > Product
15 | async function getProducts(): Promise<Product[]> {
16 |   const query = `*[_type == "product"] {
25 |   }`;
26 |
27 |   const products = await client.fetch(query);
28 |   return products;
29 | }
30 |
31 | export default async function Product() {
32 |   const products = await getProducts();
33 |
34 |   return (
35 |     <div>
36 |       <div className="p-5 max-w-7xl mx-auto">
37 |         <h1 className="text-center text-3xl font-bold mb-8">Products</h1>
38 |
39 |         <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-6">
40 |           <div>
41 |             <div>
42 |               <div>
43 |                 <div>
44 |                   <div>
45 |                     <div>
46 |                       <div>
47 |                         <div>
48 |                           <div>
49 |                             <div>
50 |                               <div>
51 |                                 <div>
52 |                                   <div>
```

Step 11: Displaying Data on the UI

Finally, I used Next.js to create a UI that dynamically rendered the fetched product data. Each product was displayed in a card layout with the name, price, description, and image.

